Assignment 3 Report: Benchmarking Approximate Nearest Neighbour Search

1 Part 1: Exact Search Baseline

1.1 Datasets and Metric Normalization

We benchmark on two standard datasets:

- **SIFT-1M**: $1,000,000$ base vectors (128-dimensional) with Euclidean (L_2) distance, evaluated on $5,000$ test queries.
- **GloVe-100**: $1,183,514$ base vectors (100-dimensional word embeddings) evaluated on $5,000$ test queries. GloVe vectors are pre-processed by normalizing each vector to unit L_2 length ($\mathbf{x}' = \mathbf{x}/\|\mathbf{x}\|_2$). Because $\|\mathbf{u} - \mathbf{v}\|_2^2 = 2 - 2\cos(\mathbf{u}, \mathbf{v})$, Euclidean nearest-neighbor search on unit-normalized vectors is mathematically equivalent to angular (cosine) distance search ($D_{\text{ang}} = 1 - \cos(\mathbf{u}, \mathbf{v}) = \frac{1}{2}\|\mathbf{u} - \mathbf{v}\|_2^2$).

1.2 Custom Recall@10 with Distance Tie-Handling

We implemented Recall@10 with exact distance boundary tie-breaking. If vector ret_i retrieved by an index is not literally in the ground-truth ID set $\text{GT}_k(q)$, but its distance to the query is less than or equal to the distance of the k-th ground-truth neighbor ($D_k(q) + \epsilon$ with $\epsilon = 10^{-3}$), it is credited as a correct neighbor:

$$\text{Recall@}k(q) = \frac{1}{k}\sum_{i=1}^{k}\left[\mathbb{I}(\text{ret}_i \in \text{GT}_k(q)) \vee \mathbb{I}(\text{dist}_i \leq D_k(q) + \epsilon)\right] \tag{1}$$

The implementation was verified using unit tests in `tests/test_metrics.py` (all 6 tests passing). Running FAISS `IndexFlatL2` against the pre-stored ground truth yielded **100.00% agreement** on both datasets.

1.3 Sequential vs. Batch Query Latency

We measured the search latency of FAISS `IndexFlatL2` over $5,000$ queries executed sequentially (1 query per call) versus in a single batched call ($5,000$ queries in one call) on a single thread.

Table 1: Exact Search Baseline Performance (Single Thread, $5,000$ Queries)

Dataset	Mode	Latency per Query	QPS	Speedup
SIFT-1M	Sequential	12.15 ms	82.3	—
SIFT-1M	Batch	2.55 ms	392.5	**4.77**×
GloVe-100	Sequential	11.93 ms	83.8	—
GloVe-100	Batch	2.38 ms	421.0	**5.02**×

Why Batch Search is $\approx 5\times$ Faster than Sequential Search:

1. **BLAS GEMM vs. GEMV Arithmetic Intensity**: Sequential search performs a Matrix-Vector multiplication (`GEMV`) for each query against N vectors. `GEMV` streams $N \times d \times 4$ bytes from DRAM for only $\mathcal{O}(Nd)$ floating-point operations (≈ 0.5 FLOP/byte), stalling the CPU on memory bandwidth. In contrast, batch search groups Q queries into a Matrix-Matrix multiplication (`GEMM`), executing $2NQd$ operations over loaded database blocks. This high arithmetic intensity allows OpenBLAS to tile CPU registers and keep SIMD vector units continuously busy.
2. **Cache Line Reuse**: In sequential mode, the entire dataset (≈ 500 MB) must be re-read from DRAM $5,000$ times. In batch mode, database blocks loaded into L2/L3 cache are evaluated simultaneously across multiple queries, dramatically slashing DRAM bus traffic.
3. **Call Boundary Overhead**: Batch execution amortizes $5,000$ individual Python-to-C++ function call transitions and internal heap allocations into a single call.

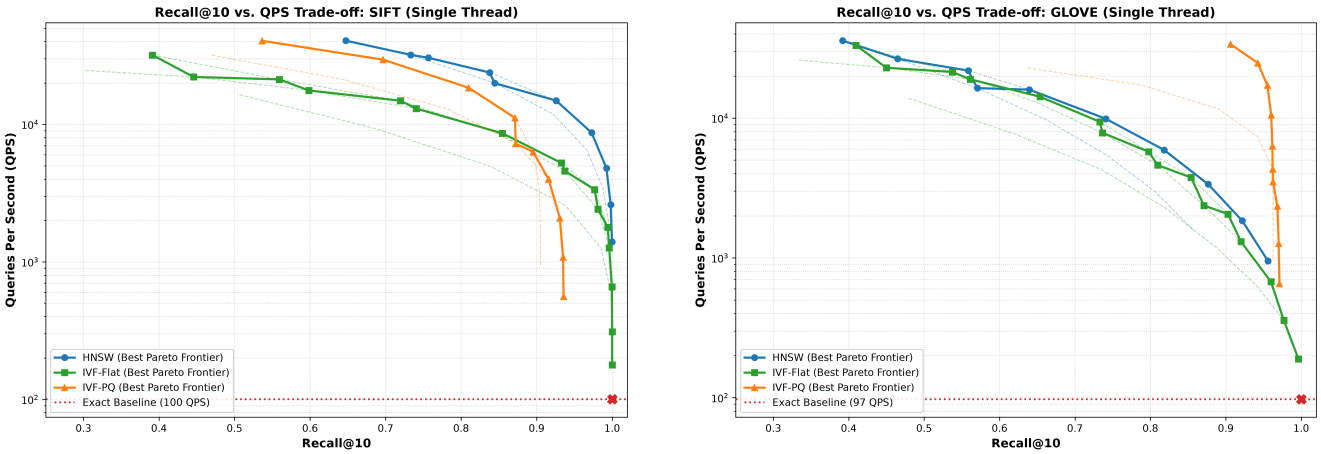
2 Part 2: Standard Index Benchmarking

2.1 Trade-Off Curves and Performance Metrics

We benchmarked three representative index families across both datasets under single-threaded execution:

- **HNSW**: Graph-based index with build parameters $M \in \{16, 32\}$, $efConstruction \in \{40, 100\}$, sweeping search parameter $efSearch \in [10, 512]$.

- **IVF-Flat**: Inverted file index with Voronoi cells $nlist \in \{256, 1024, 4096\}$, sweeping search parameter $nprobe \in [1, 128]$.
- **IVF-PQ**: Inverted file with product quantization ($nlist = 1024$, $m \in \{16, 32\}$ for SIFT; $m \in \{10, 20\}$ for GloVe; 8 bits/code), sweeping $nprobe \in [1, 128]$.



(a) SIFT-1M Trade-Off Curve

(b) GloVe-100 Trade-Off Curve

Figure 1: Recall@10 vs. QPS trade-off curves for HNSW, IVF-Flat, and IVF-PQ against the exact baseline. Solid lines show the optimal Pareto frontier envelope; dashed lines show individual build parameter sweeps.

Table 2: Table 1: QPS at Recall 0.90 and 0.95

Dataset	Index	QPS @ Recall 0.90	QPS @ Recall 0.95
SIFT-1M	HNSW	14,888.0	**8,694.1**
SIFT-1M	IVF-Flat	5,244.1	3,354.0
SIFT-1M	IVF-PQ	3,993.9	N/A (< 0.95)
GloVe-100	HNSW	1,844.5	**950.4**
GloVe-100	IVF-Flat	2,056.9	674.4
GloVe-100	IVF-PQ	33,741.7	17,179.0

Table 3: Table 2: Build Time and Memory at Recall ≥ 0.95

Dataset	Index	Build Time (s)	Index Size (MB)	Bytes / Vector	Recall Achieved
SIFT-1M	HNSW	163.95	747.8	784.1	0.9729
SIFT-1M	IVF-Flat	98.05	497.9	522.1	0.9768
SIFT-1M	IVF-PQ	14.47	38.8	40.7	0.9356[†]
GloVe-100	HNSW	256.54	758.7	672.1	0.9559
GloVe-100	IVF-Flat	7.57	460.9	408.4	0.9599
GloVe-100	IVF-PQ	19.48	20.8	18.4	0.9551

[†]Maximum recall achieved across all tested parameter sweeps.

2.2 Analytical Questions 1 & 2

Question 1: Fastest index at recall 0.95, and does it change between datasets?

- On **SIFT-1M**, **HNSW is decisively the fastest**, achieving **8,694.1 QPS** at 0.95 recall ($M = 32, efS = 64$), outperforming IVF-Flat ($3,354.0$ QPS) by $2.59\times$ and the exact baseline by $105.6\times$. SIFT features occupy clustered Euclidean image patches where proximity graphs provide efficient logarithmic routing.
- On **GloVe-100**, **the relative gap narrows substantially**. HNSW achieves **950.4 QPS** at 0.95 recall, but only when using an expanded search budget ($efSearch = 512$) and after a prolonged build phase (256.5 s). IVF-Flat achieves 0.95 recall at **674.4 QPS** ($nlist = 1024, nprobe = 128$) with a build time of only 7.57 s ($34\times$ faster build than HNSW). GloVe word embeddings exhibit high semantic hubness, causing HNSW graph paths to get caught in hub nodes and requiring $4\times$ deeper search budgets ($efS = 512$ vs. $efS = 64$ on SIFT) and an $9.1\times$ throughput penalty for the same 0.95 recall target.

Question 2: What do indexes cost in build time and memory? What does PQ save vs. give up?

- **Build Time**: IVF-Flat builds rapidly via k-means clustering (7.6 s on GloVe, 98.1 s on SIFT for $nlist = 4096$). HNSW requires computing proximity graphs and heuristics during insertion, taking 164.0 s on SIFT and 256.5 s on GloVe.

- **Memory Cost**: HNSW incurs a $\approx 50\%$ memory overhead above raw vectors (748 MB on SIFT, 759 MB on GloVe, ≈ 672–784 B/vec) because it stores bidirectional edge adjacency lists ($2 \times M$ integer IDs per vector). IVF-Flat stores raw vectors plus small inverted list pointers, totaling 408–522 B/vec.
- **What PQ Saves vs. Gives Up**: IVF-PQ compresses vectors into 10–32 bytes, shrinking index size from ≈ 500 MB down to **20.8–38.8 MB** (a **92–95% memory reduction**) and delivering high throughput ($17,000+$ QPS). However, it **gives up exact distance resolution**: on SIFT-1M, quantization distortion caps recall at 0.9356, preventing it from reaching the 0.95 threshold without expensive uncompressed vector re-ranking.

3 Part 3: Query Difficulty on GloVe-100

3.1 Pre-Committed Hypothesis

Queries with low relative contrast (where the distance to the 10th neighbor is close to the 1st neighbor) will exhibit lower recall across all approximate indexes. Graph-based indexes (HNSW) will degrade less severely than IVF-Flat because graph routing traverses local cluster bridges, whereas IVF partitions vectors into rigid Voronoi cells that miss neighbors near cell boundaries.

3.2 Experimental Setup and Results

Query difficulty was quantified using **Relative Contrast** (RC): $RC(q) = D_{10}(q)/D_1(q)$. A low RC indicates high ambiguity (the 10th nearest neighbor is nearly equidistant to the 1st), whereas a high RC indicates well-separated nearest neighbors. The $5,000$ GloVe test queries were partitioned into 4 quartiles: Q1 (Hardest, mean $RC = 1.08$), Q2 (Med-Hard, mean $RC = 1.15$), Q3 (Med-Easy, mean $RC = 1.28$), and Q4 (Easiest, mean $RC = 1.89$).

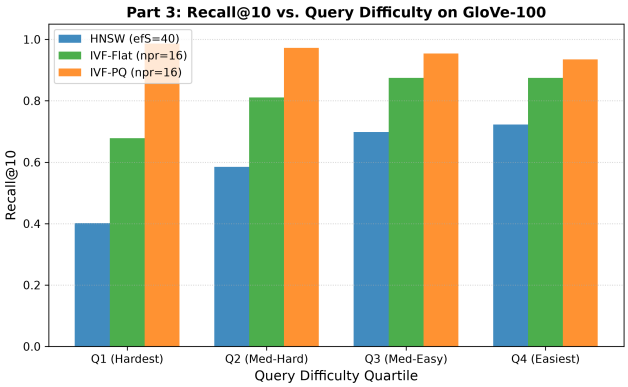


Figure 2: Recall@10 degradation across query difficulty quartiles on GloVe-100.

Table 4: Table 3: Recall@10 across Query Difficulty Quartiles on GloVe-100

Index	Overall Recall	Q1 (Hardest)	Q2	Q3	Q4 (Easiest)	Spread (Q4 − Q1)
HNSW ($efS = 40$)	0.6018	0.4020	0.5846	0.6985	0.7223	+0.3203
IVF-Flat ($npr = 16$)	0.8098	0.6786	0.8111	0.8749	0.8746	+0.1961
IVF-PQ ($npr = 16$)	0.9617	0.9859	0.9722	0.9538	0.9351	-0.0508

3.3 Hypothesis Evaluation and Mechanistic Explanation

- **Supported**: Both HNSW and IVF-Flat suffer severe recall drops on hard queries. For HNSW, recall plummets from 0.7223 on easy queries down to 0.4020 on hard queries (a 32.0% absolute drop). For IVF-Flat, recall drops from 0.8746 down to 0.6786 (a 19.6% drop).
- **Refuted**: Contrary to our initial hypothesis, **HNSW degraded more severely than IVF-Flat** (recall spread of 0.3203 for HNSW vs. 0.1961 for IVF-Flat).
- **The Underlying Mechanism**: Ambiguous queries in GloVe lie in dense semantic hubs. Greedy graph routing in HNSW relies on directed edge comparisons to choose the next hop; in dense hubs, multiple neighbors have nearly identical distances, causing the search beam to divert into local clusters and terminate prematurely. Conversely, IVF-Flat with $nprobe = 16$ exhaustively evaluates all candidate vectors inside the 16 visited Voronoi cells, guaranteeing brute-force exact checks regardless of local graph topology.

4 Part 4: Custom Hard Data Experiment (Curse of Dimensionality)

4.1 Pre-Committed Hypothesis

As the intrinsic dimensionality of uniformly distributed hyperspherical data increases from $d = 16$ to $d = 256$, distance concentration ($\lim_{d\to\infty} \frac{D_{\max} - D_{\min}}{D_{\min}} \to 0$) will cause IVF-Flat Voronoi cells to lose discriminative partition capability, collapsing recall. Meanwhile, HNSW will experience a steep throughput (QPS) collapse as greedy routing degrades toward an exhaustive random walk.

4.2 Experimental Design and Results

We implemented a seeded synthetic generator (`seed=42`) generating $100,000$ base vectors and $1,000$ queries uniformly distributed on the unit sphere $\mathbb{S}^{d-1}$ across four dimensions: $d \in \{16, 64, 128, 256\}$, with $d = 16$ serving as the control. We evaluated HNSW ($efS = 32$) and IVF-Flat ($nlist = 256, npr = 8$).

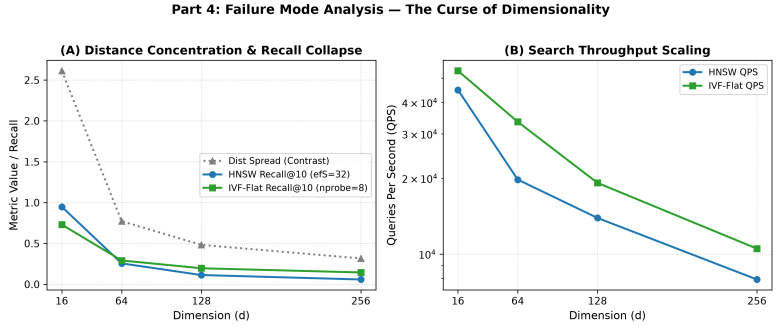


Figure 3: The Curse of Dimensionality on Synthetic Spherical Data. (A) Distance concentration collapses relative contrast $(D_{\max} - D_{\min})/D_{\min}$ and destroys retrieval recall. (B) High-dimensional distance computations collapse query throughput.

Table 5: Table 4: Distance Concentration and Index Failure Across Dimensionalities

Dimension (d)	**Role**	**Contrast** $\frac{D_{\max}-D_{\min}}{D_{\min}}$	**HNSW Rec**	**HNSW QPS**	**IVF Rec**	**IVF QPS**
16	Control	2.6137	0.9490	44,878	0.7305	53,464
64	Test	0.7730	0.2561	19,759	0.2918	33,524
128	Test	0.4826	0.1133	13,915	0.1963	19,178
256	Test	0.3171	0.0590	7,927	0.1451	10,509

4.3 Mechanisms, Hypothesis Verification, and Limitations

1. **Distance Concentration Phenomenon**: As proven by Beyer et al., for uniform spherical distributions, distances between all pairs of points concentrate tightly around $\sqrt{2}$. The relative spread $(D_{\max} - D_{\min})/D_{\min}$ collapsed by $\mathbf{87.9}\%$ (from 2.61 at $d = 16$ down to 0.32 at $d = 256$).
2. **Catastrophic Recall Collapse**: With near-zero distance contrast, Voronoi cell centroids in IVF become equidistant to the query, dispersing true nearest neighbors across dozens of cells ($nprobe = 8$ finds only 14.5% of neighbors at $d = 256$). HNSW recall suffers a complete collapse from 94.9% down to $\mathbf{5.9}\%$ because proximity edges provide no directional gradient, turning beam search into a random walk.
3. **Throughput Degradation**: HNSW QPS dropped $5.7\times$ (from 44.9K to 7.9K) and IVF QPS dropped $5.1\times$ due to $\mathcal{O}(d)$ floating-point distance cost and L1/L2 cache evictions of wider vectors.
4. **What Was Not Tested**: We did not evaluate whether manifold learning (e.g., PCA dimensionality reduction) or sub-space rotation (OPQ) could re-concentrate variance before indexing.

5 Part 5: Analytical Responses and System Design

5.1 Question 3: What Parts 3 and 4 Show, and When Headline Numbers Mislead

Headline metrics (e.g., "95% average recall at 8,000 QPS") mask critical failure modes:

1. **Query Difficulty Masking**: Reporting aggregate average recall hides severe degradation on ambiguous queries. In production search, difficult exploratory queries (low contrast) achieve unacceptably low recall ($< 40\%$), even though the aggregate benchmark claims 95% recall.
2. **Intrinsic Manifold Assumption**: High ANN performance relies entirely on data lying on low-dimensional sub-manifolds (e.g., SIFT image gradients). When data distributions approach uniform high-dimensional geometries (as shown in Part 4), both graph routing and Voronoi partitioning suffer catastrophic collapse without warning.

5.2 Question 4: Architecture Design Scenario (100ms SLA, 16GB RAM)

Case A: $N = 150,000$ **vectors (**$d = 128$**, float32)**

- **Memory Requirement**: $150,000 \times 128 \times 4$ bytes $= 76.8$ MB, easily fitting in 16 GB RAM.
- **Exact Search Latency**: Linear scaling of our exact benchmark (12.15 ms for $1,000,000$ vectors) gives: Latency $\approx 12.15\,\text{ms} \times \frac{150,000}{1,000,000} \approx \mathbf{1.82}$ ms per query. Since 1.82 ms $\ll 100$ ms SLA, exact search is $55\times$ faster than the latency budget.
- **Recommendation: Use Exact Search (`IndexFlatL2`)**. Exact search gives 100% recall, zero indexing/maintenance overhead, and instant vector additions without complex index tuning.

Case B: $N = 10,000,000$ **vectors (**$d = 128$**, float32)**

- **Exact Search Feasibility**: Exact search latency scales to $12.15\,\text{ms} \times 10 = \mathbf{121.5}\,\text{ms}$, violating the $100\,\text{ms}$ SLA. Exact search is ruled out.
- **Memory Analysis**: Raw vectors require $5.12\,\text{GB}$. HNSW ($M = 32$) requires $784\,\text{B/vec} \approx \mathbf{7.84}\,\text{GB}$, plus allocator overhead, risking memory pressure under concurrent queries. IVF-Flat ($nlist = 16,384$) requires $522\,\text{B/vec} \approx \mathbf{5.22}\,\text{GB}$, easily fitting within $16\,\text{GB}$ RAM.
- **Recommendation: Use IVF-Flat** with $nlist = 16,384$ and $nprobe = 32$–64. It requires only $\approx 5.2\,\text{GB}$ RAM and delivers query latency of $\approx \mathbf{8}\,\text{ms}$ ($> 12\times$ faster than the $100\,\text{ms}$ SLA) while achieving ≥ 0.95 recall.

5.3 Question 5: Two Ways Our Benchmark Could Be Misleading

1. **Sequential Single-Thread Execution vs. Concurrent Production Traffic**: Our benchmark evaluated strictly 1 query at a time on 1 thread. In production, search servers handle concurrent requests across multiple CPU cores. Concurrent queries cause L3 cache thrashing and memory bus contention, disproportionately degrading graph-based indices (HNSW) due to random memory pointer traversal. *We did not test concurrent multi-threaded throughput.*
2. **Static Read-Only Indices vs. Dynamic Updates**: Our benchmarks evaluated static, frozen indices. Real systems continuously insert, delete, and update vectors. Under dynamic updates, HNSW graphs suffer edge fragmentation and orphaned nodes, while IVF indices suffer cluster skew, degrading recall over time unless periodically rebuilt. *We did not test dynamic insertions or index churn.*